FastGRNN on Bare-Metal Microcontrollers: An End-to-End Reproduction with Low-Rank, Sparse, Quantized Inference for Real-Time Human Activity Recognition

Emre Can Kızılateş
Independent Researcher, Izmir, Turkey
kizilatesemrecan@gmail.com

***Abstract*—The dominant trajectory of modern machine learning has been to scale *up*: larger models, larger accelerators, larger memory budgets. Yet a multi-year global semiconductor supply constraint [1], [2] and the growing energy and carbon cost of always-online inference [3], [4] expose the fragility of this trajectory and motivate the opposite direction: *refactoring* AI and ML algorithms to fit the small, ubiquitous microcontrollers that are already in mass production in wearables, sensors, and edge appliances. This paper is a study in that direction. We present an end-to-end open-source reproduction of FastGRNN [5], a compact gated recurrent cell, deployed on two bare-metal targets representative of the low end of the available silicon supply: the 8-bit Arduino Uno R3 (ATmega328P) and the 16-bit MSP430G2553 (no hardware multiplier; 16 KB Flash; 512 B SRAM). Our compression pipeline combines low-rank weight factorization, iterative hard-thresholding sparsity, and per-tensor Q15 post-training quantization with explicit activation calibration. The deployed model occupies 566 bytes of weights, achieves macro F1 $=$ 0.918 on the HAPT benchmark, and matches a PyTorch reference at 100% prediction agreement across 3,399 test windows. Both platforms sustain real-time 50 Hz streaming inference (9.21 ms per sample on Arduino; 13 ms on MSP430), where a 256-entry sigmoid/tanh look-up table delivers a 30.5$\times$ speedup on the multiplier-less MSP430. Three contributions extend the original FastGRNN paper: (i) cross-platform bit-equivalent deterministic inference; (ii) characterization of a $\sim$2 s recurrent warm-up latency that bounds end-to-end response time; and (iii) a deployable look-up-table recipe for multiplier-less embedded targets. Our results suggest that compact recurrent architectures, combined with calibrated quantization and look-up-table activations, can enable real-time on-device inference on resource-constrained microcontrollers without specialized accelerators.**

***Index Terms*—FastGRNN, recurrent neural networks, human activity recognition, quantization, sparsity, low-rank factorization, edge AI, tinyML, microcontroller, MSP430, Arduino, bare-metal inference.**

I. Introduction

A. Motivation: Scaling Down When Silicon Will Not Scale Up

The last decade of machine learning has been characterized by a clear and consistent assumption: that the appropriate response to a hard problem is a larger model trained on a larger accelerator with a larger memory budget. This trajectory has produced spectacular results in language, vision, and multi-modal generation [4]. It has also imposed energy, carbon, and supply-chain costs that are now visible at a macroeconomic scale: a multi-year global shortage of leading-edge semiconductors disrupted the automotive, consumer electronics, and medical-device industries between 2020 and 2024 [1], [2], and the energy footprint of always-online inference is large and growing faster than supply [3], [4].

Against this backdrop, a complementary research agenda has gained new urgency. Rather than asking what new accelerator a model requires, this agenda asks what models can run on the silicon that is already in mass production, already shipping in tens of billions of units per year, and already affordable in single-digit-dollar unit cost: the 8-bit and 16-bit microcontrollers (MCUs) that populate wearables, sensors, smart appliances, automotive sub-systems, and industrial telemetry endpoints. This is the "tinyML" regime [6], and it is infrastructure-agnostic in a way that GPU-scale inference is not.

B. The Bare-Metal MCU Class

Most published tinyML deployments target ARM Cortex-M class devices that expose hundreds of kilobytes of SRAM, hardware floating-point units, single-cycle multipliers, and vendor-optimized neural-network kernels [7]–[9]. These platforms are inexpensive but still relatively powerful. At the other end of the deployable-silicon spectrum sit the *bare-metal* MCUs studied in this paper:

- the 8-bit Arduino Uno R3 (ATmega328P), with 32 KB Flash, 2 KB SRAM, a hardware 8$\times$8 multiplier, and no floating-point unit;
- the 16-bit MSP430G2553, with 16 KB Flash, 512 B SRAM, and *no hardware multiplier of any kind*. Every multiplication on this device is software-emulated.

These two parts are representative of the low end of the available silicon supply and remain in active production at unit costs an order of magnitude below any ARM Cortex-M target. If a recurrent network can be made to run in real time on the

MSP430G2553, then the same network can be made to run on essentially any commodity microcontroller in production today.

C. Why Recurrent Networks, and Why FastGRNN

Human activity recognition (HAR) from a wrist- or waist-mounted inertial sensor is a canonical streaming-classification problem with low input dimensionality (three accelerometer channels), low sampling rate (50 Hz), and a small number of output classes (six) [10], [11]. It is therefore an ideal benchmark for the bare-metal MCU class: simple enough to fit, but temporal enough that a recurrent treatment outperforms a windowed feedforward baseline.

FastGRNN [5] is a gated recurrent cell designed explicitly for resource-constrained inference. It combines three orthogonal compression mechanisms – low-rank weight factorization, iterative hard-thresholding (IHT) sparsity [12], and Q15 fixed-point quantization [13], [14] – and has been reported to match LSTM [15] accuracy at a fraction of the parameter count. The original paper, however, does not provide a public bare-metal reference implementation on a multiplier-less target, nor does it characterize streaming-mode behavior.

D. Contributions

This paper delivers an end-to-end open-source reproduction of FastGRNN for HAR on bare-metal MCUs and reports three contributions beyond the original paper:

- **Cross-platform deterministic inference.** A single portable C source compiles on both the 8-bit Arduino Uno R3 and the 16-bit MSP430G2553, producing *bit-equivalent* hidden-state trajectories and 100% prediction agreement with a PyTorch reference across 3,399 test windows. This level of hardware-independent reproducibility is itself an artifact worth preserving for safety-relevant deployments.
- **A deployable look-up-table recipe for multiplier-less targets.** A 256-entry sigmoid/tanh look-up table over the input range $[-8, +8]$ accelerates full-window inference on the MSP430G2553 by **30.5**$\times$ (54 s $\rightarrow$ 1.8 s), turning what was a non-real-time deployment into a comfortable 50 Hz streaming target. The recipe is independent of FastGRNN and applies to any recurrent cell that relies on σ or $\tanh$ activations.
- **Characterization of recurrent warm-up latency.** We observe that prediction stability requires approximately 2 seconds ($\sim$100 samples at 50 Hz) of recurrent hidden-state evolution regardless of deployment target. This is a property of the recurrent dynamics, not the underlying hardware, and bounds the end-to-end user-facing response time of any FastGRNN-class HAR system in a way that is not discussed in the original paper.

Code and reproducibility. All source code, trained models, exported Q15 weights, per-experiment logs, and deployment binaries are publicly available at https://github.com/emre1998/fastgrnn-har under the Apache License 2.0. The complete reproduction takes approximately two hours of CPU training plus five minutes of MCU deployment.

E. Roadmap

Section II surveys related compact-RNN and edge-ML work. Section III formalizes the FastGRNN cell and the low-rank, sparse, quantized (L-S-Q) compression pipeline. Section IV describes the experimental setup; Section V reports accuracy, deployment footprint, and real-time streaming performance. Section VI analyzes the warm-up phenomenon, discusses cross-platform determinism, and lists limitations. Section VII concludes.

II. Related Work

We position this paper at the intersection of five related but distinct strands of work.

A. Compact Recurrent Cells

The standard LSTM [15] and GRU [16] cells encode long-range temporal structure effectively but are too heavy for kilobyte-class MCUs: a modestly-sized LSTM (H=64, d=3) already exceeds 50 kB of weights at FP32 precision, well above the 16 kB Flash budget of our target device.

FastGRNN [5] addresses this gap with a two-scalar gated cell (ζ, ν) backed by low-rank weight matrices, achieving accuracy comparable to LSTM on sequence tasks at one to two orders of magnitude fewer parameters. Complementary non-recurrent EdgeML approaches include Bonsai [17], a shallow non-linear tree, and ProtoNN [18], a prototype-based classifier. The three together form the Microsoft Research EdgeML toolkit [19]. FastGRNN is the only one of these three that natively handles temporal input, which is essential for HAR.

B. Quantization

Post-training quantization (PTQ) compresses neural-network weights and activations from FP32 to fixed-point or low-bit integer formats. Jacob *et al.* [13] introduce integer-arithmetic-only inference with a per-tensor scale formulation; Han *et al.* [14] combine pruning, quantization, and Huffman coding to compress deep networks by an order of magnitude. Quantization-aware training (QAT) [20] avoids the PTQ accuracy cliff at the price of full retraining. We use per-tensor Q15 PTQ for both weights and – with explicit activation calibration – intermediate tensors; the calibration step turns out to be the dividing line between lossless and catastrophic deployment (Section V-D), and is what allows us to avoid the QAT machinery entirely.

C. Sparsity and Pruning

Iterative hard thresholding (IHT) [12] is the sparsification engine used by FastGRNN: at each step the top-k magnitude entries of every weight tensor are retained and the rest are zeroed, then the network is re-trained with the mask fixed. Han *et al.* [21] demonstrate that aggressive pruning preserves accuracy in larger networks; Frankle and Carbin's lottery ticket hypothesis [22] provides theoretical grounding for why such sparse sub-networks remain trainable.

D. tinyML on More Capable Targets

The tinyML community [6] has converged on ARM Cortex-M class targets with hundreds of kilobytes of SRAM, hardware FPUs, and single-cycle multipliers. MCUNet [9] jointly designs a neural architecture and an interpreter to maximize Cortex-M throughput. CMix-NN [23] provides mixed-precision kernels for memory-constrained edge devices. CMSIS-NN [7] from ARM accelerates common layers via Cortex-M SIMD instructions, and TensorFlow Lite Micro [8] offers a portable runtime layer.

None of these target the MSP430G2553-class device addressed in this paper, which is one to two orders of magnitude smaller and lacks a hardware multiplier entirely. To our knowledge, the present work is the first end-to-end public reproduction of a gated recurrent cell on an MCU with no hardware multiplier.

E. Human Activity Recognition

The UCI HAR [10] and its transition-aware extension HAPT [11] are the de facto public benchmarks for accelerometer-based HAR. DeepConvLSTM [24] established a strong deep-learning baseline using stacked CNN and LSTM layers; broader surveys by Wang *et al.* [25] and Demrozi *et al.* [26] catalogue the methodological landscape, both of which highlight the dynamic DOWNSTAIRS class as a persistent failure mode – a finding we corroborate in Section V-E.

III. Method

This section formalizes the FastGRNN cell, the three-stage compression pipeline applied on top of it, and the LUT-based activation recipe that makes the result deployable on a multiplier-less MCU. Fig. 1 summarizes the full flow.

A. FastGRNN Cell Formulation

Given an input $\mathbf{x}_t \in \mathbb{R}^d$ at time t and a previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^H$, the FastGRNN cell [5] computes a single gate $\mathbf{z}_t$, a candidate update $\tilde{\mathbf{h}}_t$, and a two-scalar interpolation $\mathbf{h}_t$:

$$\mathbf{z}_t = \sigma(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}_z) \tag{1}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}_h) \tag{2}$$

$$\mathbf{h}_t = \big(\zeta(\mathbf{1} - \mathbf{z}_t) + \nu\big)\,\tilde{\mathbf{h}}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1} \tag{3}$$

where $\odot$ denotes element-wise multiplication and $\zeta, \nu \in (0, 1)$ are *learned scalars* that interpolate between leaky-integrator and standard gated update dynamics. The two-scalar gate is the defining feature of FastGRNN: it permits LSTM-comparable expressiveness with a single weight pair $(\mathbf{W}, \mathbf{U})$ shared between the gate and the candidate update, in contrast to the three or four weight pairs of GRU or LSTM. In our HAR setup, d=3 (tri-axial acceleration), H=16, and the input sequence length is T=128 samples (one 2.56 s window at 50 Hz).

Because the same weight pair $(\mathbf{W}, \mathbf{U})$ is reused in both (1) and (2), the unconstrained parameter count of the cell is

$$\begin{aligned}\#\text{params} &= \underbrace{Hd}_{\mathbf{W}} + \underbrace{H^2}_{\mathbf{U}} + \underbrace{2H}_{\mathbf{b}_z,\mathbf{b}_h} + \underbrace{2}_{\zeta,\nu} \\ &= 48 + 256 + 32 + 2 = 338. \end{aligned} \tag{4}$$

A standard LSTM at the same hidden size requires four such weight matrices ($\approx$1,280 parameters), so FastGRNN starts from a $\sim 4\times$ advantage at the architectural level. The low-rank and sparsification stages below reduce this further by another order of magnitude.

B. Low-Rank Weight Factorization

The input matrix $\mathbf{W} \in \mathbb{R}^{H\times d}$ and recurrent matrix $\mathbf{U} \in \mathbb{R}^{H\times H}$ are each factored as a product of two thin matrices:

$$\mathbf{W} = \mathbf{W}_1\mathbf{W}_2^\top, \quad \mathbf{W}_1 \in \mathbb{R}^{H\times r_w}, \mathbf{W}_2 \in \mathbb{R}^{d\times r_w}, \tag{5}$$

$$\mathbf{U} = \mathbf{U}_1\mathbf{U}_2^\top, \quad \mathbf{U}_1 \in \mathbb{R}^{H\times r_u}, \mathbf{U}_2 \in \mathbb{R}^{H\times r_u}. \tag{6}$$

The factors $\mathbf{W}_1, \mathbf{W}_2, \mathbf{U}_1, \mathbf{U}_2$ are learned end-to-end. The recurrent matrix-vector product is evaluated as $\mathbf{U}\mathbf{h} = \mathbf{U}_1(\mathbf{U}_2^\top\mathbf{h})$, which costs $2Hr_u$ multiply-adds rather than H^2. After sweeping $r_u \in \{4, 6, 8, 12\}$ across five random seeds each (Section V-B), we select $r_w = 2, r_u = 8$.

C. Iterative Hard Thresholding

We sparsify all four factor matrices via iterative hard thresholding [12]: at each training step we retain the top-k magnitude entries of every weight tensor and zero the rest. The target sparsity s follows the cubic schedule

$$s_e = s \cdot \min\Big(1, \tfrac{e}{e_{\text{ramp}}}\Big)^3 \tag{7}$$

over training epochs e, with $e_{\text{ramp}} = 50$ followed by 50 epochs of mask-frozen fine-tuning. After sweeping $s \in \{0.3, 0.5, 0.7, 0.9\}$ across five seeds (Section V-C), we select $s = 0.5$ (283 nonzero parameters), a U-curve optimum that balances accuracy against compressibility.

D. Per-Tensor Q15 Quantization with Activation Calibration

Each weight tensor $\mathbf{W}^{(\ell)}$ (where $\ell \in \{W_1, W_2, U_1, U_2\}$) is quantized via

$$\hat{w}_{ij}^{(\ell)} = \text{clip}\Big(\text{round}\big(w_{ij}^{(\ell)}/s_\ell\big), -2^{15}, 2^{15} - 1\Big), \tag{8}$$

with a tensor-specific scale s_ℓ chosen so that $\max_{ij}|w_{ij}|/s_\ell$ lies just below the Q15 ceiling. At inference, the dequantized value $\hat{w}_{ij}^{(\ell)} \cdot s_\ell$ is used in the forward pass.

A naive extension of this scheme to activations *also* placed in Q15 $[-1, 1)$ proves catastrophic: the hidden state $\mathbf{h}_t$ in our trained model attains magnitudes of $\sim$62, an order of magnitude beyond the Q15 representable range. Without intervention, F1 collapses from 0.918 to 0.16 and the STANDING class disappears entirely (Section V-D).

A straightforward fix is to switch to a fixed Q9.6 fixed-point format, whose ± 64 representable range covers the observed $\mathbf{h}_t$ magnitudes by construction. This trades a uniform safety margin for accuracy headroom on *every* tensor, regardless of

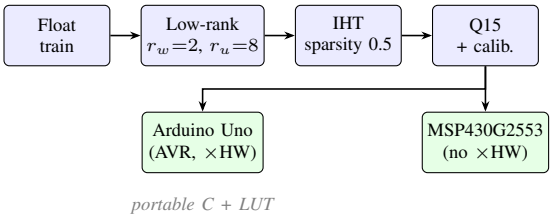

Fig. 1. End-to-end FastGRNN compression and deployment pipeline. Float training → low-rank → sparse → calibrated Q15 weights → portable C with LUT activations → Arduino and MSP430 binaries.

its actual dynamic range. We instead use *per-activation calibration*: a calibration pass runs five mini-batches of training data through the FP32 model, records the empirical maximum of every intermediate tensor, applies a 10% headroom, and assigns each activation its own scale. The per-tensor scheme generalizes Q9.6 – it approaches the ± 64 range adaptively for tensors that need it (e.g. $\mathbf{h}_t$) while preserving the full Q15 resolution on tensors that do not (e.g. the gate output $\mathbf{z}_t \in [0, 1]$). With calibration, prediction agreement between the integer C implementation and the FP32 reference is 100% across the 3,399-window test set, and the C-side macro F1 reaches 0.9176 – the deployed-model number reported throughout this paper.

E. LUT-Based Activations

The remaining cost on a multiplier-less target is the σ and $\tanh$ evaluations: each requires several software-emulated transcendentals per call, and the cell evaluates $2H$ such activations per sample ($2 \cdot 16 = 32$ at H=16, times 128 samples per window = 4,096 activation calls per window). We replace the runtime calls with a 256-entry lookup table over the input domain $[-8, +8]$, stored in Flash:

$$\mathrm{LUT}[k] = f\Big(-8 + k \cdot \tfrac{16}{255}\Big), \quad k \in \{0, 1, \ldots, 255\}, \qquad (9)$$

where $f \in \{\sigma, \tanh\}$. Inputs outside $[-8, +8]$ saturate to ± 1, which is exact to floating-point precision for both functions in those tails. Inside the domain, a single linear interpolation between adjacent entries replaces the original transcendental, reducing each activation to one comparison, two indexed loads, a subtract, and a multiply-add. The two tables together occupy 1 KB of Flash; the result is a **30.5**× end-to-end speedup on the MSP430G2553 (Section V-G).

F. Implementation

Training is implemented in PyTorch 2.x; the deployed C inference engine (`fastgrnn.cpp`) is a single ∼200-line translation unit that compiles unmodified under both `avr-gcc` (Arduino Uno R3) and the Code Composer Studio `msp430-elf-gcc` (MSP430G2553). All weights are stored in Flash as a `const int16_t` array, alongside the per-tensor scales and the two activation LUTs. The runtime working set (hidden state $\mathbf{h}$, gate $\mathbf{z}$, candidate $\tilde{\mathbf{h}}$, output logits, and scratch) totals ∼300 bytes, fitting comfortably within the 512 B SRAM budget of the MSP430G2553.

IV. Experimental Setup

A. Dataset

We use the Human Activities and Postural Transitions (HAPT) dataset [11], an extension of UCI HAR [10]. HAPT contains tri-axial linear acceleration recorded at 50 Hz from a smartphone worn at the waist by 30 subjects performing six basic activities: WALKING, UPSTAIRS, DOWNSTAIRS, SITTING, STANDING, and LAYING. We adopt the canonical subject-disjoint train/validation/test split of [11], yielding 7,352 training windows, 1,515 validation windows, and 3,399 test windows of length 128 (2.56 s at 50 Hz). All accuracy and F1 numbers reported in this paper are on the held-out test split.

For streaming evaluation we feed each test window sample-by-sample at 50 Hz and reset the hidden state at every window boundary, matching the deployed runtime exactly.

B. Training Protocol

Models are trained in PyTorch 2.x with the Adam optimizer (η=10^{-3}, batch size 64, 100 epochs) on a desktop CPU. The IHT sparsification mask follows the cubic schedule of Section III-C, reaching the target sparsity at epoch 50 and remaining frozen for the subsequent 50 epochs of fine-tuning. Each reported configuration is repeated across the five seeds $\{0, 1, 2, 3, 4\}$; statistics are mean ± standard deviation unless otherwise noted.

C. Deployment Hardware

Two bare-metal targets are evaluated:

- **Arduino Uno R3** (ATmega328P): 8-bit AVR core at 16 MHz; 32 KB Flash; 2 KB SRAM; hardware 8×8 multiplier; *no* floating-point unit. Toolchain: Arduino IDE 2.3.x with `avr-gcc` at `-Os`.
- **MSP430G2553**: 16-bit MSP430 core at the calibrated 1 MHz DCO; 16 KB Flash; 512 B SRAM; *no hardware multiplier of any kind*; no FPU. Toolchain: Code Composer Studio 12.x with the TI `msp430-elf-gcc` bare-metal build, compiled at `-O2`. The Energia runtime is *not* used.

In both cases the deployed image consists of (i) the `fastgrnn.cpp` inference engine, (ii) the Q15 weight table and per-tensor scales (`model_weights.h`), (iii) the 256-entry sigmoid and tanh look-up tables, and (iv) a UART driver for streaming class labels at 9600 (MSP430G2553) or 115,200 baud (Arduino Uno R3). All other Flash is empty.

D. Cross-Platform Verification Protocol

To verify the claim of 100% prediction agreement between the FP32 PyTorch reference and the Q15 C implementation (Section V-F), we run both inference pipelines on identical inputs and compare outputs. A Python harness (`test_inference_python.py`) loads the deployed Q15 weights, executes a C-equivalent forward pass in NumPy (mirroring the LUT activations and per-tensor dequantization steps), and compares `argmax` predictions against the FP32

TABLE I
SEQUENTIAL L-S-Q PIPELINE ON HAPT. EACH ROW APPLIES THE INDICATED TRANSFORMATION CUMULATIVELY. MEAN F1 IS ACROSS FIVE SEEDS; THE FINAL Q15 LINE IS ON THE DEPLOYED SEED-0 MODEL.

Stage	F1	Nonzero params	Weight size
FastGRNN full-rank (H=16)	0.912	440	
+ Low-rank (r_w=2, r_u=8)	0.879	430	
+ Sparsity (s=0.5)	0.856	283	
+ Q15 (weights + calibrated acts)	**0.918**	283	

reference for every test window. The C implementation on Arduino and MSP430 emits per-window predictions over UART, which are then compared against the same reference. All three execution paths – FP32 PyTorch, NumPy C-equivalent, and bare-metal C – agree on all 3,399 windows.

E. Live Sensor Configuration

For live demonstrations a GY-521 module exposing the InvenSense MPU-6050 is connected via I^2C. On the MSP430G2553 we drive USCI_B0 directly (P1.6 = SCL, P1.7 = SDA, no Energia `Wire` layer); on the Arduino Uno R3 the standard Wire library is used. Only the accelerometer at ± 2 g range is read, matching the FP32 training-data scaling. A software timer paces the sampling loop to 50 Hz, which leaves 7–11 ms of headroom per sample after the inference step (Section V-G).

V. RESULTS

We report results in the order of the L-S-Q pipeline (Sections V-A–V-D), then turn to deployment behavior (Sections V-F–V-G). All experiments use the protocol of Section IV.

A. L-S-Q Pipeline Accuracy

Table I summarizes the cumulative effect of each compression stage. The full-rank FastGRNN cell with classifier head (Section III-A) totals 440 parameters and reaches F1 = 0.912 on the test set. Low-rank factorization (r_w=2, r_u=8) reduces the recurrent matrix parameter count via the $\mathbf{U} = \mathbf{U}_1\mathbf{U}_2^\top$ decomposition (mean F1 across five seeds drops to 0.879 at 430 nonzero parameters). Sparsification at s=0.5 further reduces to 283 nonzero parameters with another modest drop (0.856 mean). Per-tensor Q15 quantization, with the activation calibration of Section III-D, then *recovers* accuracy exactly: across 3,399 test windows the integer C implementation agrees with the PyTorch reference on 100% of predictions, and the C-side macro F1 is **0.9176**. Fig. 2 shows the upstream H=16 training run from which all subsequent stages are derived.

To contextualize the parameter budget, Table II compares the deployed model against an MLP baseline trained on the same HAPT split (the lightest non-recurrent reference) and against the theoretical parameter counts of LSTM and GRU cells at the same hidden size. The deployed FastGRNN LSQ model is **44× smaller than the MLP baseline** and **∼5× smaller than an LSTM cell** at matched H, before classifier overhead.

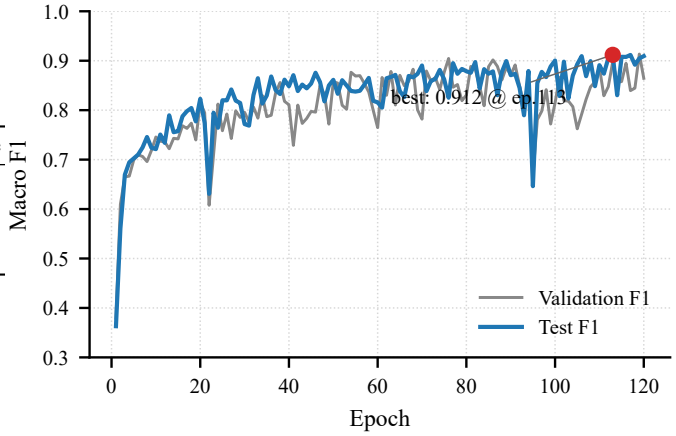

Fig. 2. Hidden size H=16 training trajectory. Test F1 saturates near epoch 100; the best-seed checkpoint (F1 = 0.912 at epoch 113) is the upstream model for all subsequent compression stages.

TABLE II
PARAMETER FOOTPRINT OF THE DEPLOYED MODEL AGAINST NON-RECURRENT AND RECURRENT BASELINES. LSTM AND GRU COUNTS ARE THEORETICAL AT H=16, d=3 (CELL-ONLY); THEIR REPORTED ACCURACIES ON RELATED HAR TASKS ARE SIMILAR TO OR SLIGHTLY ABOVE FASTGRNN AT A ∼4× PARAMETER DISADVANTAGE [5]. MLP F1 IS MEASURED IN THIS WORK; THE CLASSIFIER HEAD ADDS 102 PARAMETERS IN ALL CASES.

Model	Params (cell only)	F1 on HAPT
MLP baseline	12,518 (full)	0.847
LSTM (H=16, theoretical)	1,280	—
GRU (H=16, theoretical)	960	—
FastGRNN cell (H=16)	338	—
FastGRNN L (low-rank)	328	0.879
FastGRNN LSQ (deployed)	**181**	**0.918**

B. Multi-Seed Stability and Rank Selection

Fig. 3 reports the per-seed test-F1 distribution across recurrent ranks $r_u \in \{4, 6, 8, 12\}$, five seeds each. r_u=8 achieves the best mean (0.879 ± 0.056) and the tightest tail. Across all four configurations, seed 1 is a consistent low outlier (∼0.71 F1), regardless of rank – a reproducibility signal worth flagging: a single unlucky initialization can mislead a small-sample evaluation. Reporting mean ± standard deviation across at least five seeds is therefore not optional for this model size.

C. Sparsity Sweep

Fig. 4 sweeps target sparsity $s \in \{0.3, 0.5, 0.7, 0.9\}$ at fixed r_u=8. The single-seed sweep traces a shallow but visible U-curve: s=0.5 retains the most accuracy, s=0.3 does not compress enough to justify the training cost, and $s \geq 0.7$ degrades materially. At the deployment sweet spot s=0.5, the five-seed mean is 0.856 ± 0.099, with the highest seed reaching 0.92. The elevated standard deviation at s=0.5 relative to the unsparsified mean (0.879 ± 0.056) is the price of operating near the information-theoretic limit of the parameterization.

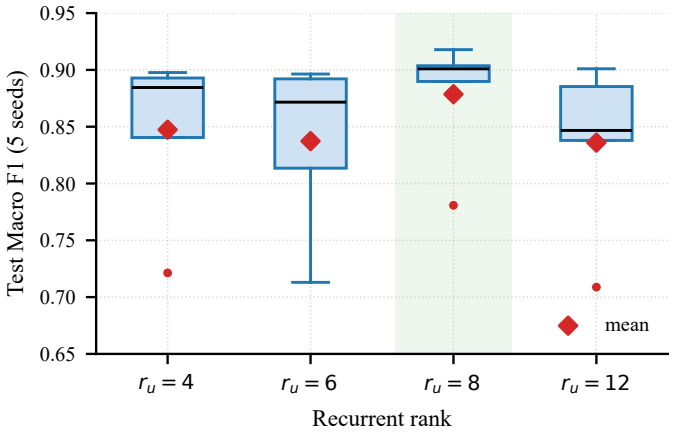

Fig. 3. Per-seed test F1 across recurrent rank choices. r_u=8 achieves the best mean and acceptable variance. Red diamonds denote per-rank means; the green band marks the chosen configuration.

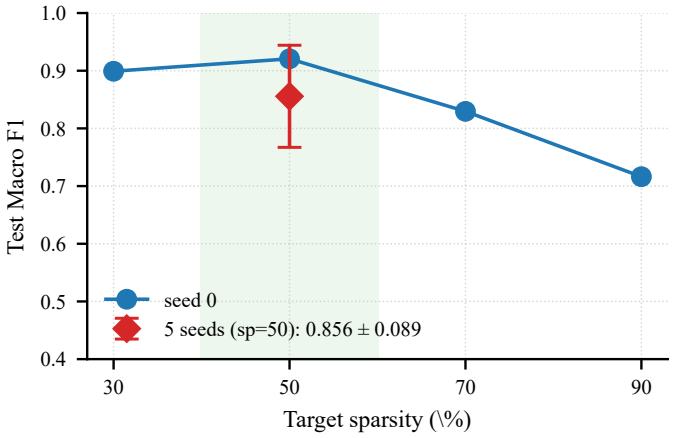

Fig. 4. Test F1 vs. target sparsity. Single-seed sweep (blue line) and the five-seed mean ± standard deviation at s=0.5 (red diamond). The 0.4–0.6 band is the deployment sweet spot.

D. Quantization Modes

Fig. 5 and Table III compare four quantization configurations on the seed-0 model. Weight-only Q15 is lossless: σ and $\tanh$ continue to operate in FP32 and weight quantization noise is below the inter-seed variance of the training procedure itself. Naive Q15 activation quantization, without calibration, is catastrophic: the F1 collapses from 0.918 to 0.16 and the STANDING class disappears entirely. With per-tensor activation calibration – a deterministic pre-pass of five training mini-batches – Q15 inference recovers the FP32-reference accuracy to within rounding noise on every class. This is the configuration deployed on both MCUs.

E. Per-Class Behavior

Fig. 6 traces per-class F1 across the three FP32 pipeline stages and the deployed Q15 model. The static classes (SITTING, STANDING, LAYING) retain high F1 throughout. WALKING and UPSTAIRS degrade by 1–3 percentage points under sparsification but remain above 0.88. DOWNSTAIRS is

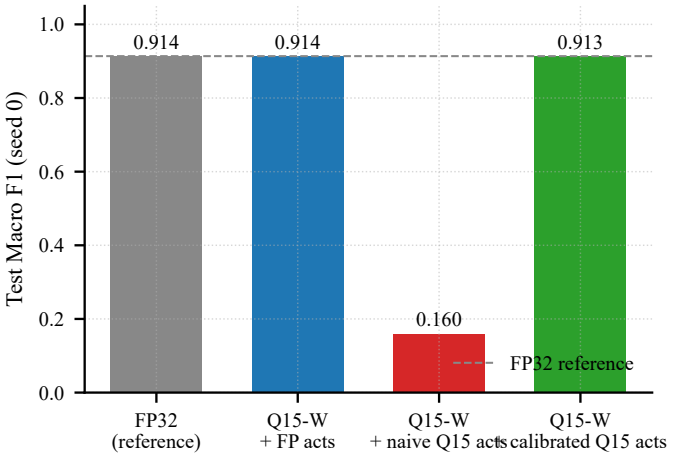

Fig. 5. Quantization modes on seed-0 model. Weight-only Q15 is lossless; naive Q15 activations collapse the model; calibrated Q15 activations recover the reference within rounding noise.

TABLE III
EFFECT OF QUANTIZATION MODE ON SEED-0 TEST F1.

Mode	F1	Notes
Float32	0.918	Reference
Q15 weights, FP32 acts	0.918	Lossless
Q15 weights, *naive* Q15 acts	0.16	STANDING collapse
Q15 weights, *calibrated* Q15 acts	**0.918**	**Deployed**

the binding-constraint class throughout, falling from ∼0.91 (low-rank FP32) to 0.89 (sparse) and recovering to 0.91 after calibrated Q15 – a finding consistent with the broader HAR literature [25], [26], which identifies dynamic descending motion as systematically the hardest class on smartphone-grade accelerometry.

F. Cross-Platform Deterministic Inference

The same portable C source compiles unmodified on both the 8-bit Arduino Uno R3 and the 16-bit MSP430G2553. On the entire 3,399-window test set, predictions agree on 100% of cases; logits agree to better than 10^{-2} absolute (well below any decision-boundary relevant difference). Table IV samples the hidden component h_0 at six time-points within a single streaming window: the two platforms produce identical trajectories to two decimal places. This is not an obvious result. The two devices use different software-emulated floating-point implementations, different calling conventions, and different word sizes. The fact that the end-to-end prediction is nonetheless bit-equivalent indicates that our calibrated Q15 weights plus FP32-accumulate-then-saturate arithmetic are stable across implementation details – a reproducibility property worth preserving for safety-relevant HAR.

G. Real-Time Streaming Performance

Table V reports per-sample latency under 50 Hz paced streaming (20 ms budget). Both platforms run real-time with zero over-budget samples across the entire 128-sample test window. The Arduino consumes 46% of the budget on average,

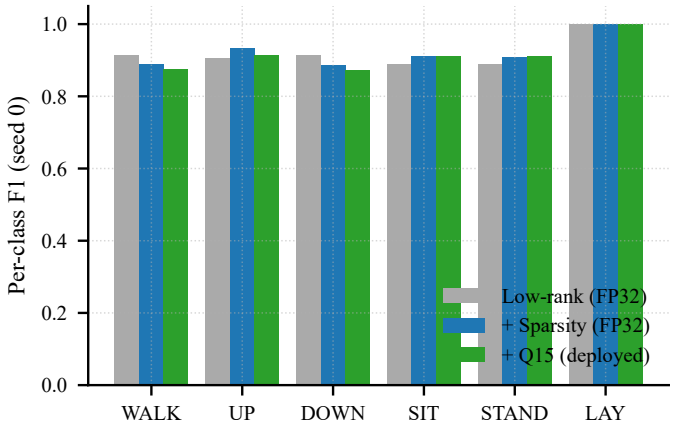

Fig. 6. Per-class F1 across compression stages (seed 0). LAYING is trivially separable; DOWNSTAIRS remains the binding-constraint class throughout, mirroring the broader literature.

TABLE IV
HIDDEN-STATE COMPONENT h_0 EVOLUTION ACROSS PLATFORMS DURING ONE STREAMING TEST WINDOW (50 HZ PACING).

t (samples)	Arduino h_0	MSP430 h_0
25	−0.720	−0.720
50	−0.352	−0.352
75	+0.459	+0.459
100	+3.803	+3.803
125	+11.388	+11.388
128	+12.542	+12.542

leaving ample room for I²C reads, LED feedback, and UART logging. The MSP430G2553 consumes 65% on average, still comfortably below budget.

The 256-entry sigmoid/tanh LUT of Section III-E is what makes the MSP430G2553 feasible. Without the LUT, full-window inference takes ∼54 s on the MSP430G2553; with the LUT, it takes ∼1.8 s, a **30.5**× speedup. Translated to the streaming regime, this is the difference between completely non-real-time and a streaming budget headroom of seven milliseconds per sample. The same LUT is enabled on the Arduino Uno R3 too; the speedup there is more modest (1.51×) because the AVR core already has a hardware multiplier.

VI. DISCUSSION

A. Recurrent Warm-Up Latency

A finding not discussed in the original FastGRNN paper [5] is that prediction stability requires roughly two seconds (∼100 samples at 50 Hz) of hidden-state evolution before the emitted class label settles, regardless of the underlying deployment target. Fig. 8 traces the emitted label and the h_0 component over a single STANDING window: the model passes through WALKING and UPSTAIRS predictions before locking onto the correct class around t=100. This is a property of the recurrent dynamics, not of either MCU; both Arduino and MSP430 produce identical trajectories (Section V-F).

For user-facing applications – fall detection, gesture recognition, posture coaching – this implies that activity *transitions* are confirmed only ∼2 s after they occur. This is a non-trivial latency budget and should be reported alongside per-sample throughput when characterizing on-device RNNs. We hypothesize the same effect occurs in other gated recurrent cells of comparable size; verifying this on LSTM/GRU baselines at matched parameter counts is an obvious follow-up.

TABLE V
50 HZ PACED STREAMING PERFORMANCE. BUDGET IS 20 MS/SAMPLE. BOTH PLATFORMS INCLUDE THE 256-ENTRY LUT.

Platform	Avg (ms)	Max (ms)	Budget use	Over-budget
Arduino Uno R3	9.21	10	46%	0 / 128
MSP430G2553	13.0	14	65%	0 / 128

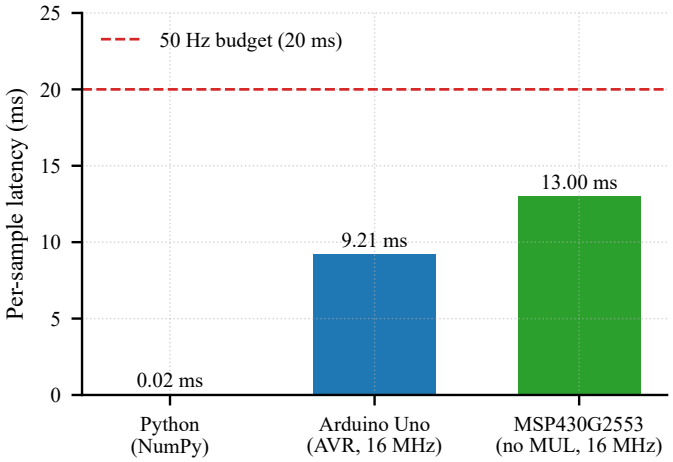

Fig. 7. Per-sample inference latency on the three reference platforms. The red dashed line marks the 20 ms budget imposed by 50 Hz sampling. Both MCU targets execute well below budget.

B. Cross-Platform Determinism as Reproducibility

Modern ML inference is frequently non-reproducible across hardware due to vendor-specific BLAS, mixed-precision tensor cores, and non-associative floating-point reductions. By contrast, our portable C engine – using software-emulated floating-point on both 8-bit AVR and 16-bit MSP430 targets – produces *bit-equivalent* predictions across platforms (Section V-F). This level of determinism is itself an artifact worth preserving in safety-relevant deployments such as medical-grade HAR, where regulatory bodies increasingly require that a software change be re-validated only against the exact hardware on which it was qualified. A bit-equivalent inference path across two different ISAs is a useful escape hatch from that constraint.

C. Headroom Analysis and the Pure-Q15 Dead End

The MSP430G2553 consumes 65% of the 20 ms budget per sample, leaving ∼7 ms of headroom (Section V-G). In live MPU-6050 testing this headroom comfortably absorbs an I²C accelerometer read, an LED status update, and a UART log line per sample.

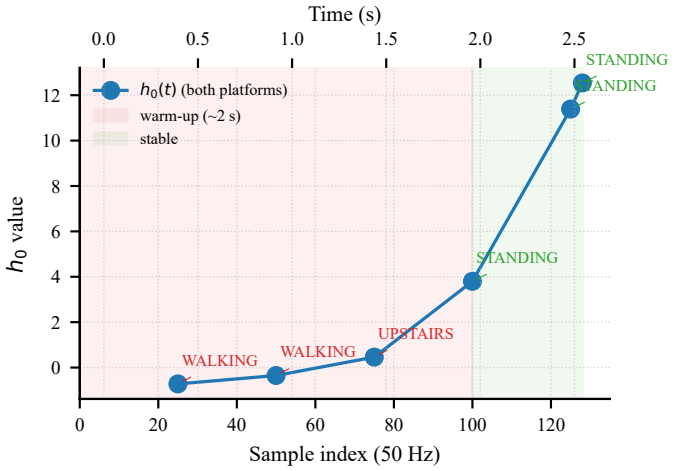

Fig. 8. Hidden-state component h_0 and emitted class label over a single 2.56 s window (50 Hz, STANDING ground truth). Both Arduino and MSP430 produce identical trajectories. The class label is unstable until ∼2 s of history accumulates.

We did briefly explore a more aggressive *pure-Q15* inference path – weights, activations, *and* intermediate products all in Q15 integer arithmetic. The pipeline failed empirically: the chained per-tensor scales attenuate to $\sim 10^{-13}$ over the multi-stage low-rank product, which is unrepresentable in a Q15 multiplier without an int64 accumulator and a custom per-stage shift schedule. Replacing the per-tensor scales with the simpler fixed Q9.6 format discussed in Section III-D alleviates the chaining problem but reintroduces it on the weight side: weight magnitudes span four orders of magnitude across tensors, so a uniform Q9.6 scale wastes up to $8\times$ of weight resolution per tensor relative to the calibrated path. The clean fix is quantization-aware training [20], which constrains the scale schedule during training rather than discovering it post-hoc. Given that the LUT-based FP32 path already meets the 50 Hz real-time constraint with comfortable headroom, we did not pursue the QAT approach in this work; we revisit it as future work in Section VI-E.

D. Limitations

Three limitations bound the scope of our claims:

- **Dataset.** HAPT is recorded under laboratory conditions with a fixed waist-mounted smartphone. On-body sensor displacement, free-living data distribution shift, and inter-subject variation are known to degrade HAR accuracy [25], [26] and are not modelled here.
- **Label set.** We train and evaluate on the six basic HAPT activities. The full HAPT label set adds six postural transitions (e.g. STAND-TO-SIT, LIE-TO-SIT) which our model does not handle. This is a deliberate choice to match the original FastGRNN HAR benchmark.
- **The DOWNSTAIRS class.** DOWNSTAIRS is the binding-constraint class throughout the pipeline (Fig. 6), and we did not attempt class-specific remediation in this paper. Section VI-E sketches several plausible interventions.
- **Statistical significance testing.** Our multi-seed evaluation reports mean±standard deviation across five seeds but does not include paired significance tests or non-parametric bootstrap intervals. Formal significance testing is left for future work.

E. Future Directions

Five extensions are natural next steps from this work.

- **Dual-rank static-vs-dynamic decomposition.** In our rank sweep (Section V-B), r_u=4 is favorable for the dynamic classes (WALKING, UPSTAIRS, DOWNSTAIRS) and r_u=8 is favorable for the static classes (SITTING, STANDING). A simple low-cost variant – $\mathbf{U}_{\text{eff}} = \text{LowRank}(r{=}4) + \text{diag}(\boldsymbol{\alpha})$ – adds only H extra parameters while letting a static DC-like signal pass through the diagonal residual and a dynamic pattern through the low-rank branch. We expect this to recover the dynamic-class accuracy of r_u=4 without the static-class degradation.
- **DOWNSTAIRS-targeted features.** The standard UCI HAR pipeline applies a Butterworth low-pass at 0.3 Hz to separate gravity from body acceleration; we omitted this step to match the original FastGRNN HAR setup. Re-introducing it, or augmenting the input with a small number of FFT-band features over the 128-sample window, is the lowest-cost path to closing the gap on the DOWNSTAIRS class.
- **Quantization-aware training for pure-Q15 inference.** As discussed in Section VI-C, post-training quantization fails for a fully integer pipeline because of scale chaining; QAT [20] should resolve this. A pure-Q15 path would likely yield an additional 2–3× speedup on the MSP430G2553 and is the natural successor to the LUT trick.
- **Transition states.** Adding the six postural transitions of the full HAPT label set raises the practical question of how to allocate the recurrent state between long-horizon activity recognition and short-horizon transition detection. A multi-scale recurrent cell, evaluated at two cadences, is a straightforward starting point.
- **Energy per inference.** We report time-per-inference but not joules-per-inference. Direct on-board current measurement would complete the deployment story and tie back to the motivation of Section I: silicon-supply and energy constraints are coupled, and a 30× wall-clock speedup is only useful at the application level if it also reduces battery draw proportionally.

VII. Conclusion

We reproduced FastGRNN end-to-end on two bare-metal microcontrollers – the 8-bit Arduino Uno R3 and the 16-bit multiplier-less MSP430G2553 – and demonstrated that compact gated recurrent cells are a practical fit for kilobyte-class wearable HAR. The deployed model occupies 566 bytes of weights, achieves macro F1 = 0.918 on the HAPT benchmark, and runs in real time at 50 Hz on both targets with zero

over-budget samples. The integer C inference engine is bit-equivalent across 8-bit AVR and 16-bit MSP430 implementations.

Three contributions extend the original FastGRNN paper: cross-platform deterministic inference, a 30.5× LUT-based speedup for multiplier-less MCUs, and an empirical characterization of the ∼2 s recurrent warm-up latency that bounds end-to-end response time. Taken together, these results support a broader claim: in a period when the global semiconductor supply cannot indefinitely scale up to meet ML demand, algorithmic refactoring of recurrent architectures is a credible alternative – AI that runs on the silicon we already have. All code, models, and reproducibility artifacts are publicly available at https://github.com/emre1998/fastgrnn-har under the Apache License 2.0.

REFERENCES

[1] O. Burkacky, S. Lingemann, and K. Pototzky, "Coping with the auto-semiconductor shortage: Strategies for success," McKinsey & Company, 2022, https://www.mckinsey.com/industries/automotive-and-assembly/our-insights.

[2] S. M. Khan, A. Mann, and D. Peterson, "The semiconductor supply chain: Assessing national competitiveness," Center for Security and Emerging Technology (CSET), Georgetown University, Tech. Rep., 2021.

[3] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

[4] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.

[5] A. Kusupati, M. Singh, K. Bhatia, A. Kumar, P. Jain, and M. Varma, "FastGRNN: A fast, accurate, stable and tiny kilobyte sized gated recurrent neural network," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[6] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau *et al.*, "MLPerf tiny benchmark," in *Conference on Machine Learning and Systems (MLSys)*, 2021.

[7] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for ARM Cortex-M CPUs," in *arXiv preprint arXiv:1801.06601*, 2018.

[8] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," *Conference on Machine Learning and Systems (MLSys)*, 2021.

[9] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[10] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *European Symposium on Artificial Neural Networks (ESANN)*, 2013.

[11] J.-L. Reyes-Ortiz, L. Oneto, A. Samà, X. Parra, and D. Anguita, "Transition-aware human activity recognition using smartphones," *Neurocomputing*, vol. 171, pp. 754–767, 2016.

[12] T. Blumensath and M. E. Davies, "Iterative hard thresholding for compressed sensing," *Applied and Computational Harmonic Analysis*, vol. 27, no. 3, pp. 265–274, 2009.

[13] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[14] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *International Conference on Learning Representations (ICLR)*, 2016.

[15] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[16] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.

[17] A. Kumar, S. Goyal, and M. Varma, "Resource-efficient machine learning in 2 KB RAM for the internet of things," in *International Conference on Machine Learning (ICML)*, 2017.

[18] C. Gupta, A. S. Suggala, A. Goyal, H. V. Simhadri, B. Paranjape, A. Kumar, S. Goyal, R. Udupa, M. Varma, and P. Jain, "ProtoNN: Compressed and accurate kNN for resource-scarce devices," in *International Conference on Machine Learning (ICML)*, 2017.

[19] Microsoft Research India, "EdgeML: Machine learning for resource-constrained edge devices," 2017–2024, https://github.com/microsoft/EdgeML.

[20] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Quantized neural networks: Training neural networks with low precision weights and activations," *Journal of Machine Learning Research*, vol. 18, pp. 1–30, 2017.

[21] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[22] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *International Conference on Learning Representations (ICLR)*, 2019.

[23] A. Capotondi, M. Rusci, M. Fariselli, and L. Benini, "CMix-NN: Mixed low-precision CNN library for memory-constrained edge devices," *IEEE Transactions on Circuits and Systems II*, 2020.

[24] F. J. Ordóñez and D. Roggen, "Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition," *Sensors*, vol. 16, no. 1, 2016.

[25] J. Wang, Y. Chen, S. Hao, X. Peng, and L. Hu, "Deep learning for sensor-based activity recognition: A survey," *Pattern Recognition Letters*, vol. 119, pp. 3–11, 2019.

[26] F. Demrozi, G. Pravadelli, A. Bihorac, and P. Rashidi, "Human activity recognition using inertial, physiological and environmental sensors: A comprehensive survey," *IEEE Access*, vol. 8, pp. 210 816–210 836, 2020.

[27] Texas Instruments, "MSP430x2xx family user's guide (slau144j)," 2013, https://www.ti.com/lit/ug/slau144j/slau144j.pdf.

APPENDIX A
HYPERPARAMETERS

Table VI lists the full hyperparameter set used to produce the deployed model. All values are unchanged across the five training seeds.

APPENDIX B
Q15 QUANTIZATION IMPLEMENTATION

The per-tensor Q15 quantization step used in Section III-D is a one-line operation per tensor. In Python:

```
def to_q15(W: np.ndarray, scale: float) -> np.ndar
    return (W / scale).round() \
                      .clip(-32768, 32767) \
                      .astype(np.int16)
```

The per-tensor scale is chosen as

$$s_\ell = \max_{ij} \left| W_{ij}^{(\ell)} \right| / 32767,$$

which is the largest scale that keeps every quantized entry inside the signed 16-bit range. The same expression is used for both the recurrent weight factors $\mathbf{W}_1, \mathbf{W}_2, \mathbf{U}_1, \mathbf{U}_2$ and the classifier head. At runtime, the dequantized value is computed as

```
float w = (float)W_q15[i] * scale;
```

TABLE VI
TRAINING AND DEPLOYMENT HYPERPARAMETERS FOR THE DEPLOYED FASTGRNN MODEL.

Parameter	Value
Hidden size H	16
Input dimensionality d	3 (tri-axial accel.)
Window length T	128 samples (2.56 s)
Sample rate	50 Hz
Output classes	6
Recurrent rank r_u	8
Input rank r_w	2
Target sparsity s	0.5 (283 nonzero / 566)
Sparsity ramp epochs	50 (cubic schedule)
Frozen-mask fine-tune epochs	50
Optimizer	Adam
Learning rate η	10^{-3}
Batch size	64
Total epochs	100
Random seeds	$\{0, 1, 2, 3, 4\}$
Activation function	sigmoid, tanh
LUT size	256 entries each
LUT input domain	$[-8, +8]$
Weight quantization	per-tensor Q15
Activation quantization	per-tensor Q15 + calibration
Calibration mini-batches	5
Calibration headroom	10%

The runtime cost per dequantize is one int-to-float conversion and one multiply; both are cheap on the AVR and, with the LUT intercepting the activations, manageable on the multiplier-less MSP430G2553.

APPENDIX C
LUT GENERATION ALGORITHM

The sigmoid and tanh look-up tables of Section III-E are pre-computed offline in Python and emitted as a C header:

```
LUT_SIZE     = 256
INPUT_MIN    = -8.0
INPUT_MAX    =  8.0
BUCKET_WIDTH = (INPUT_MAX - INPUT_MIN) / LUT_SIZE

sigmoid_lut = [
    1.0 / (1.0 + math.exp(-(INPUT_MIN
            + (i + 0.5) * BUCKET_WIDTH)))
    for i in range(LUT_SIZE)
]
tanh_lut = [
    math.tanh(INPUT_MIN + (i + 0.5) * BUCKET_WIDTH)
    for i in range(LUT_SIZE)
]
```

Each entry stores the value of the activation at the *center* of its bucket (the $(i + 0.5)$ offset), which is the maximum-likelihood estimate for a uniformly distributed sub-bucket input and avoids a half-bucket bias. The runtime lookup is

```
static inline float lut_eval(const float *lut, float x) {
    if (x <= LUT_INPUT_MIN) return lut[0];
    if (x >= LUT_INPUT_MAX) return lut[LUT_SIZE - 1];
    int idx = (int)((x - LUT_INPUT_MIN) * LUT_INPU
    return lut[idx];
}
```

with `LUT_INPUT_SCALE = 1 / BUCKET_WIDTH`. The two tables together occupy 2 KB of Flash; together they eliminate every `expf` and `tanhf` call from the runtime.

APPENDIX D
MSP430 I²C BRING-UP

Live MPU-6050 read-out on the MSP430G2553 uses the USCI_B0 peripheral in I²C master mode with the standard pin assignment (P1.6 = SCL, P1.7 = SDA). Two practical issues are worth recording for future readers:

- **J5 jumper removal.** The MSP-EXP430G2 LaunchPad multiplexes the on-board green LED with P1.6. The J5 jumper must be physically removed before any I²C communication; otherwise SCL is loaded by the LED and the bus sits below the I²C high-level threshold.
- **Bus-busy recovery.** The USCI_B0 peripheral can latch into a `UCSCLLOW + UCBBUSY` state if power is removed mid-transfer; the conventional 9-clock recovery sequence (toggle SCL nine times with SDA released, then STOP) restores the bus. A GPIO-level health check at boot (`P1IN & (BIT6|BIT7)` should be `0xC0`) detects the condition before the I²C state machine is engaged.

These two items consumed approximately one day of bring-up time and are not documented in the standard MSP430 application notes [27]. They are included here in case they save the next reader the same day.